



File

File e Dati

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio



File

File?



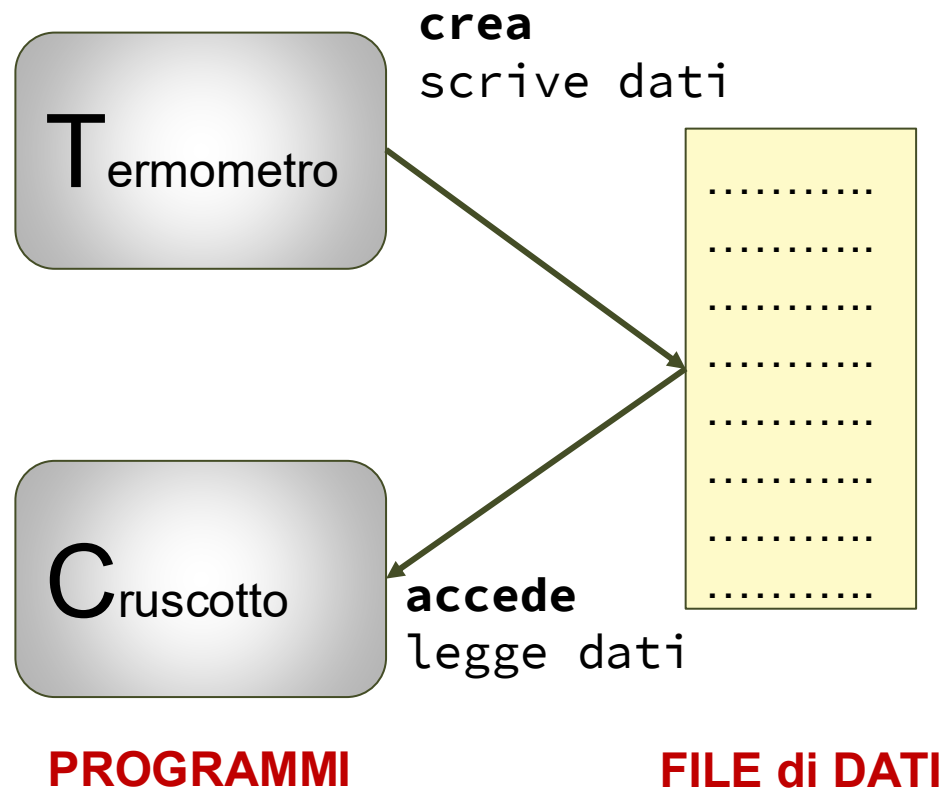
- File:
 - **Unità** in cui, agli occhi dell'utente, è organizzata la **memoria persistente**¹ di un computer.
- Un file contiene una collezione di dati correlati, ad esempio:
 - Immagini (.jpg, .gif, .png, ...)
 - Video (.mov, ...)
 - Testi e programmi (.txt, .c, .java, ...)
 - Queste slide
 - Dati

¹Persistente: che sopravvive allo spegnimento del computer

Programmi che Elaborano File: Esempi

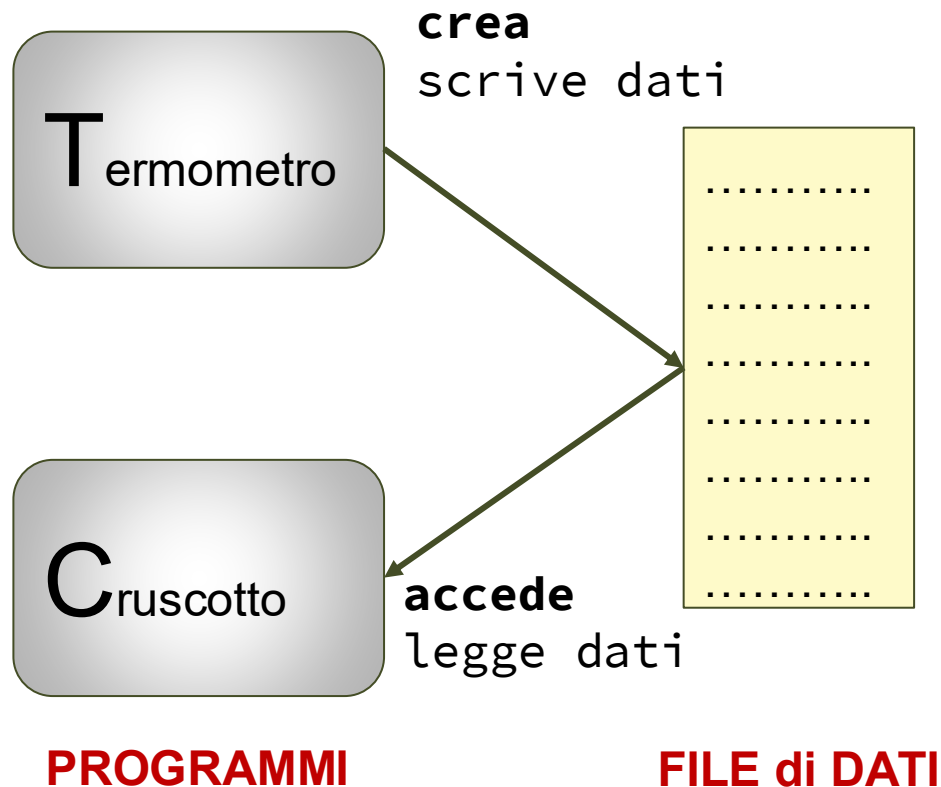
- Un **compilatore**, e.g. gcc,
 - È un programma che prende in input uno o più file sorgenti e produce un file nuovo
- Un **editor**
 - È un programma che può creare un file nuovo o modificarne uno esistente
- Una **galleria di immagini**
 - È un programma che permette di scorrere e visualizzare file
- Un video **player**
 - È un programma che riproduce file video
- Un **browser**
 - È un programma che mostra file .html

Caso di Studio: Termometro + Cruscotto



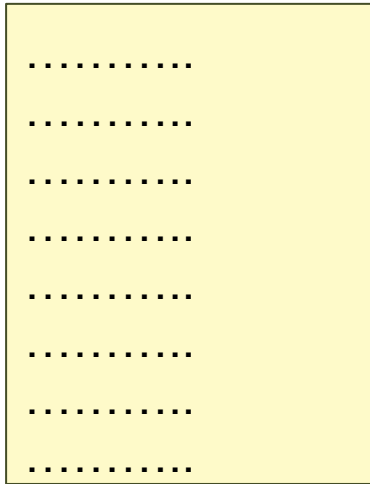
- Due semplici programmi:
 - **Termometro** registra in un file le temperature delle ore 12:00 di tutti i giorni della settimana
 - Produttore di dati in file
 - **Cruscotto** legge tali valori e calcola temperatura media, minima e massima della settimana
 - Consumatore dei dati in file
- Domande:
 - Come si crea un file?
 - Come si accede a un file?
 - Come si legge da un file?
 - Come si scrive in un file?

Architettura di Termometro + Cruscotto



- I **file** sono **risorse**
 - 3 fasi: acquisizione, uso, e rilascio
 - Simile alla memoria dinamica via `malloc`, `uso` e `free`
- Un programma deve:
 1. Ottenere l'accesso
 2. Usare il file
 3. Rilasciare il file

File e Dati



FILE di DATI

- I **dati** in un file sono organizzati secondo un **formato**
 - Esempio
 - Numeri interi separati da spazi
- Per elaborare i dati è necessario conoscere il formato
 - Termometro e cruscotto **condividono** il formato

I/O Formattato: Funzioni

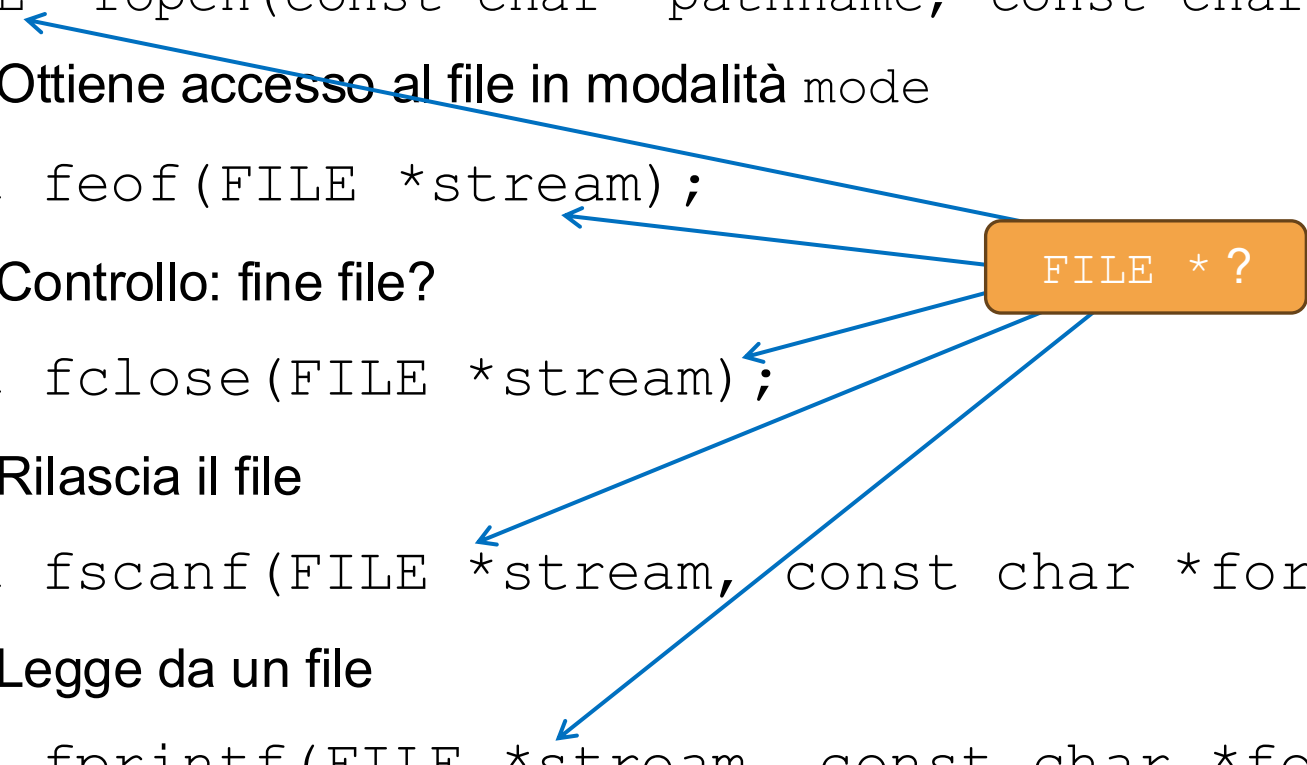
I/O Formattato su File: Funzioni

- Cinque funzioni per input/output (I/O) formattato su file
- `fopen`
 - Ottiene accesso al file di un certo tipo
- `fscanf`
 - Legge da un file
- `feof`
 - Controllo: fine file?
- `fprintf`
 - Scrive in un file
- `fclose`
 - Rilascia il file

Standard I/O

- Contenute nel modulo **stdio**: `#include <stdio.h>`
- `FILE *fopen(const char *pathname, const char *mode)`
 - **Ottiene accesso al file in modalità** `mode`
- `int feof(FILE *stream);`
 - **Controllo: fine file?**
- `int fclose(FILE *stream);`
 - **Rilascia il file**
- `int fscanf(FILE *stream, const char *format, ...)`
 - **Legge da un file**
- `int fprintf(FILE *stream, const char *format, ...)`
 - **Scrive in un file**

FILE * ?



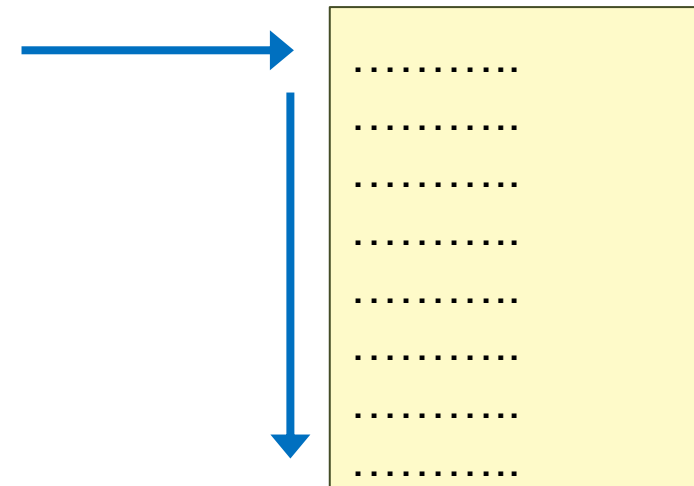
Il Tipo: **FILE** *

- **FILE** è un tipo di dato definito in `stdio.h`
 - Rappresenta un **flusso** di dati da o verso un file
- `fopen` apre un file in una modalità (`r`, `w`, ...)
 - L'apertura produce un flusso di dati (`FILE*`)
 - La **direzione** del flusso (da o verso il file) **dipende dalla modalità** di apertura (`r`, `w`, ...)
 - Esempio:
 - `fopen("file", "w")` produce un flusso di dati verso il file che non può essere usato per leggere i contenuti



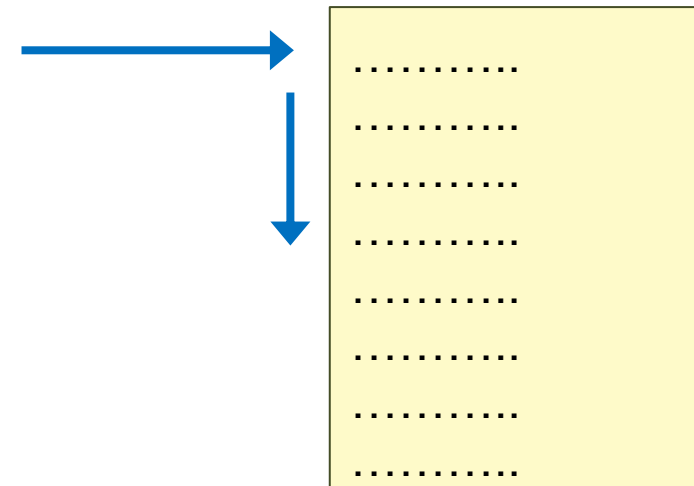
Apertura: `fopen` `r` (read)

- Apertura in **lettura** (**read**):
 - Posizione del FILE*:
 - Primo byte del file
- Lettura **sequenziale**:
 - Ogni lettura sposta il puntatore automaticamente del numero di byte letti



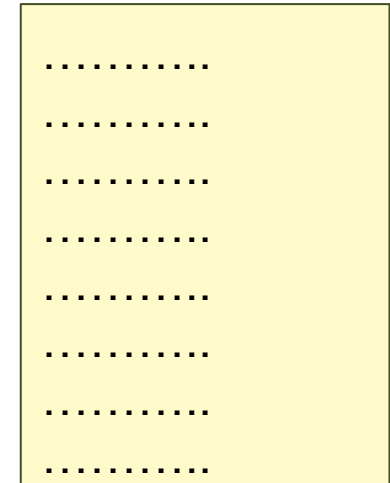
Apertura: `fopen` `w` (write)

- Apertura in **scrittura** (**write**):
 - Posizione del FILE*:
 - Primo byte del file
- Scrittura **sequenziale**:
 - Ogni scrittura sposta il puntatore automaticamente del numero di byte letti
 - Eventuali contenuti precedenti vengono cancellati



Rilascio: `fclose`

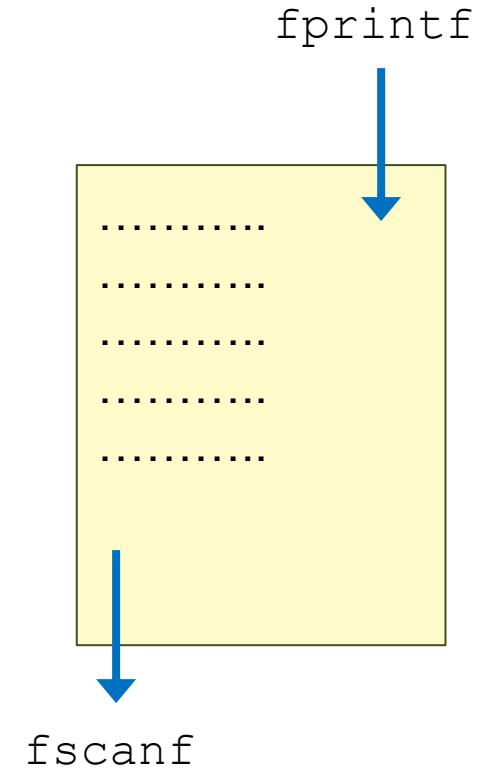
- Un file aperto deve essere rilasciato
- I file sono risorse:
 - Vanno richiesti (`fopen`)
 - E obbligatoriamente rilasciati (`fclose`)
- `fclose` ha per argomento il `FILE*` da chiudere
 - Non chiudere il file può creare inconsistenza dei dati
 - Il programmatore deve ricordarsi di invocare `fclose` nei punti opportuni del programma



Lettura/scrittura: fscanf/fprintf

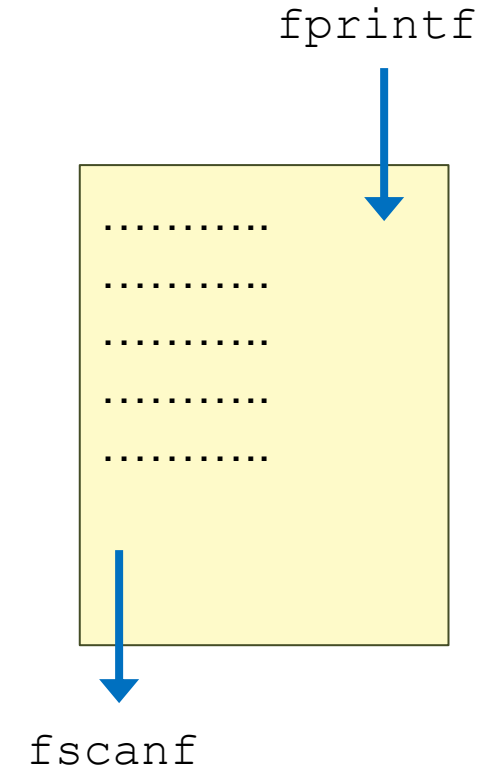


- **Lettura/scrittura:**
 - `fscanf` (lettura) e `fprintf` (scrittura) permettono di operare su un `FILE*`
- Entrambe prevedono la **specificazione** di un **formato**
 - Una stringa che indica cosa leggere/scrivere
 - Il formato è come per `printf/scanf`
 - Seguono ulteriori parametri in numero variabile



Fine?: `feof`

- **Test:**
 - I file hanno lunghezza variabile
 - Quando leggiamo occorre uno strumento per capire se siamo giunti alla fine del file
 - `feof` è un test su un `FILE*`
 - `eof` = end of file



Esempi

Termometro: Produttore dei Dati



```
#include <stdio.h>
[...]  
FILE * fp = fopen("dati.txt", "w");
```

Modalità di accesso del file:

- “w” : apertura file in scrittura

`fp` permetterà tutte le operazioni sul file:

- In caso di errore:
 - `fopen` restituisce `NULL`

pathname del file:

- Se esiste:
 - File viene “aperto” cioè reso accessibile
- Se non esiste:
 - File viene creato

Esempio: Scrittura di un Numero Intero



```
#include <stdio.h>
```

```
[...]
```

```
int x;
```

```
[...]
```

```
FILE * fp = fopen("dati.txt", "w");
```

```
fprintf(fp, "%d", x);
```

```
fclose(fp);
```

Apertura in scrittura

Scrittura di x nel file

Chiusura del file (rilascio delle risorse)

Cruscotto: Consumatore dei Dati



```
#include <stdio.h>
[...]  
FILE * fp = fopen("dati.txt", "r");
```

`fp` permetterà tutte le operazioni sul file:

- In caso di errore:
 - `fopen` restituisce `NULL`

Modalità di accesso del file:

- “r” : apertura file in lettura

pathname del file:

- Se esiste:
 - Viene “aperto” cioè reso accessibile
- Se non esiste:
 - Errore e restituisce `NULL`

Esempio: Lettura di un Numero Intero

```
#include <stdio.h>
```

```
[...]
```

```
int x;
```

```
[...]
```

```
FILE * fp = fopen("dati.txt", "w");
```

```
fscanf(fp, "%d", &x);
```

```
fclose(fp);
```

Apertura in lettura

Lettura dal file: x conterrà il
valore letto

Chiusura del file (rilascio delle
risorse)

Un Pattern per Cruscotto



- Struttura Cruscotto in sei sezioni:
 1. Definizione variabile le operazioni sul file
 2. Apertura file
 3. Gestione errore
 4. Uso del file
 - Più dettagli nelle prossime slides
 5. Chiusura file
 6. Eventuale altro codice
 - Esempio: stampa a video risultato

```
#include <stdio.h>
int main() {
    FILE *fp;
    // eventuali altre variabili

    fp = fopen("dati.txt", "r");
    if (fp == NULL) { ... return ...; }

    // uso il file

    fclose(fp);

    // altro codice
}
```

Un Pattern per Cruscotto: Uso File I

- Uso file:
 - Leggiamo finché ci sono dati
 - Che condizione?
 - `feof` per controllo condizione fine del file
- `feof` restituisce
 - Valore diverso da 0 una volta raggiunto l'ultimo byte del file
 - Altrimenti 0: che in una condizione equivale a falso

```
if (fp != NULL) {  
    while (!feof(fp)) {  
        // finché non ho esaurito i  
        contenuti  
  
        ... leggi dato  
  
        ... elabora dato  
  
    }  
  
    fclose(fp);  
}
```

Un Pattern per Cruscotto: Uso File II

- Uso file:
 - Leggo dato
 - Come?
 - `fscanf` per lettura e variabile interna `temp`
- Dopo lettura:
 - `temp` conterrà il valore letto dal file
 - `fp` viene aggiornato automaticamente

```
if (fp != NULL) {  
    while (!feof(fp)) {  
        // finché non ho esaurito i  
        contenuti  
  
        fscanf(fp, "%d ", &temp);  
  
        ... elabora dato  
  
    }  
  
    fclose(fp);  
  
}
```


Un Pattern per Cruscotto: Uso File III



- Uso file:
 - Elaboro dato
 - Calcolo minimo e massimo
 - `min` e `max` nuove variabili
 - Inizializzate come?
 - `min = MAX_VALUE`
 - `max = MIN_VALUE`
- Media:
 - Cosa serve?

```
if (fp != NULL) {  
    while (!feof(fp)) {  
        // finché non ho esaurito i  
        contenuti  
  
        fscanf(fp, "%d ", &temp);  
        if (temp<min) min=temp;  
        if (temp>max) max=temp;  
        // media?  
    }  
  
    fclose(fp);  
}
```

Un Pattern per Cruscotto: Uso File IV



- Media basata su:

$$\bar{v} = \frac{1}{N} \sum_{i=0}^N v$$

- Serve tenere traccia di:
 - Conteggio valori letti
 - Somma valori letti
- Infine:
 - Stampa risultato a video

```
if (fp != NULL) {  
    while (!feof(fp)) {  
        // finché non ho esaurito i  
        contenuti  
  
        fscanf(fp, "%d", &v);  
        if (temp<min) min=v;  
        if (temp>max) max=v;  
        cont++; sum+=v;  
    }  
  
    fclose(fp);  
    printf("min=%d, max=%d,avg=%d",  
        min, max, sum/cont);  
}
```

Termometro: Simulazione



- Termometro:
 - Simulato da utente che inserisce sette temperature
- Sezioni:
 1. Apertura file in scrittura
 2. Scrittura temperatura letta
 - Ripetuta 7 volte
 3. Chiusura file
- Osservazioni?

```
#include <stdio.h>

int main() {
    int temp; int i;
    // apertura file in scrittura
    FILE *fp = fopen("dati.txt", "w");
    for (i=0;i<7;i++) {
        // simuliamo lettura temperatura per 7 giorni
        printf("%d, temp? ", i);
        scanf("%d", &temp);

        fprintf(fp, "%d ", temp); // scrive nel file
    }
    fclose(fp);
}
```

Esempio Termometro e Cruscotto: Note



- Il codice di Termometro è incompleto
 - Manca la gestione del NULL, completate voi
 - Quando può fallire l'apertura in scrittura?
 - Esempio: spazio su disco finito
- Come si potrebbe generalizzare il termometro in modo che, a ogni esecuzione, possa inserire un numero di temperature o di cicli diverso?
 - Esempio: all'avvio chiedo quanti valori leggere da tastiera
 - Il cruscotto va modificato?
 - No: cruscotto legge un dato alla volta fino alla fine del file
 - Ignoriamo problemi di overflow

- `fopen` ha numerose altre modalità
 - Esempio: “a” che permette l’apertura in “append”
 - Il puntatore si posiziona in fondo ai contenuti attuali per aggiungere dati a quelli già esistenti
- `fseek` permette di muovere il puntatore di lettura e/o scrittura all’interno del file
 - `int fseek(FILE *fp, long distanza, int partenza)`
 - `distanza` indica di quanti byte il file pointer deve essere spostato
 - `partenza` indica da quale posizione deve essere spostato:
 - `SEEK_SET` – inizio del file
 - `SEEK_END` – fine del file
 - `SEEK_CUR` – posizione corrente

Esempio Riassuntivo



- Obiettivo:
 - Scrivere un programma che, a scelta dell'utente, scrive su oppure legge da uno stesso file
- Coniuga:
 - Interazione con utente (input da tastiera, output su terminale)
 - Con input/output su file

Esempio Riassuntivo: Scheletro Scelta

```
#include <stdio.h>
#define MAX 20
```

```
int main() {
    // Il programma deve, a scelta dell'utente, leggere o scrivere un file
    int scrivi;
    FILE *fp; char datiS[MAX]; int datiN;

    printf("\nVuoi leggere o scrivere? 1: scrivere, 0: leggere ");
    scanf("%d", &scrivi);

    if (scrivi) { // 1: vero, 0: falso
        ...
    }
    else {
        ...
    }
}
```

Scelta

Esempio Riassuntivo: Scrittura vs Lettura



```
{
...
if (scrivi) {
    fp = fopen("dati.txt", "w"); // chiedo accesso in w al file
    if (fp) { // se ottenuto
        fprintf(fp, "nome: Anna id: 123\n"); // scrivo
        fclose(fp); // rilascio il file
    }
}
else {
    fp = fopen("dati.txt", "r"); // chiedo accesso in r al file
    if (fp) { // se ottenuto
        fscanf(fp, "nome: %s id: %d\n", datiS, &datiN); // leggo
        printf("\nHo letto:%s %d\n", datiS, datiN);
        fclose(fp); // rilascio il file
    }
}
}
```

Scrittura

Lettura

Possibile
problema?

Lettura Formattata



```
fscanf(fp, "nome: %s id: %d\n", datiS, &datiN);
```

- Il formato di lettura specifica che ci aspettiamo che la stringa e il numero da leggere siano inseriti in un testo che segue il pattern:
 - nome: <NOME-DA-LEGGERE> id: <NUM-DA-LEGGERE>
- La parte di pattern esplicito non viene memorizzata
 - Cosa succede se i dati non corrispondono al pattern atteso?
 - Provate più esecuzioni modificando i dati/formato

Esercizio



- Scrivere un programma che:
 - Legge dati da un primo file contenente cifre scritte come numeri (0, 1, 2 ... 9)
 - E produce un secondo file contenente le stesse cifre scritte come parole (zero, uno, due, ... nove).
- Tutto il contenuto del primo file (opportunamente codificato) deve essere riversato nel secondo file
- Lasciato da fare a voi